

# learn-dev

Plateforme web d'apprentissage de la  
programmation

Eric Bouchut — Titre professionnel Développeur  
Web et Web Mobile (DWWM)

# INTRODUCTION

# Le besoin

- Apprendre à programmer suppose un **parcours structuré** : du contenu organisé, une progression lisible, un espace où un formateur publie et fait évoluer ses cours.
- **Problème** : offrir un environnement multi-rôles simple, **sûr** et **accessible**.
- **Solution, en une phrase** :  
*learn-dev est une plateforme web où des formateurs publient des cours composés de leçons en Markdown, et où des étudiants s'inscrivent et les suivent.*

## learn-dev en bref

**Trois rôles** : étudiant (STUDENT), formateur (INSTRUCTOR), administrateur (ADMIN).

**Étudiant** : catalogue, inscription à un cours, lecture des leçons.

**Formateur** : création de cours/leçons, cycle de vie de publication, gestion des inscrits.

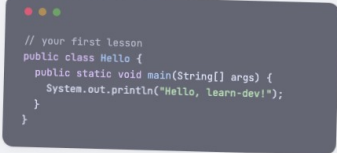
**Administrateur** : gestion des comptes, modération des contenus.

[learn-dev](#)[Home](#)[Log in](#)[Sign up](#)

## Learn programming by doing

Interactive lessons, hands-on exercises, and instant feedback.  
Build real skills at your own pace, one lesson at a time.

[Create your account](#)[I already have one](#)



```
// your first lesson
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, learn-dev!");
    }
}
```

### Why learn-dev?



#### Interactive lessons

Read, try, and experiment in the same place. Every concept comes with editable examples.



#### Instant feedback

Automated assessment tells you what works and what to fix, the moment you submit.



#### Track your progress

Courses, lessons, and streaks: watch your skills grow on your personal dashboard.

[learn-dev](#) — an interactive programming learning platform.[Source on GitHub](#)

**CONCEPTION**

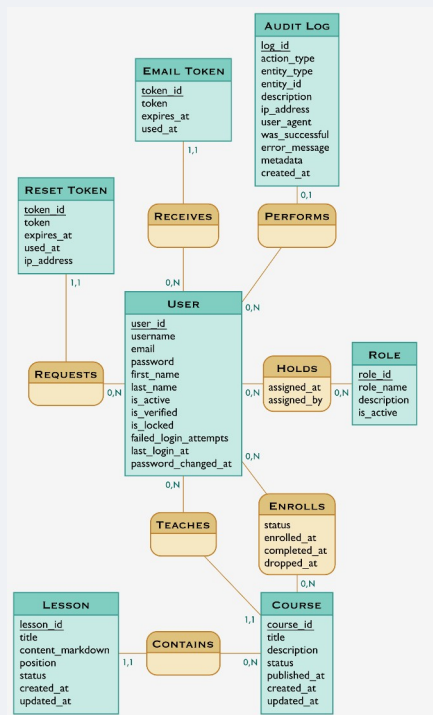
# Gestion de projet

- **Git par fonctionnalité** : une branche par feature, convention *Conventional Commits*.
- **Développement incrémental** : auth → domaine cours → authoring → administration.
- **14 décisions d'architecture (ADR)** tracées (format MADR).
- Principe **YAGNI** : on ne construit que le nécessaire à la v1.

# Cahier des charges & personas

- **Objectifs** : s'inscrire et suivre des cours, publier/gérer des cours, administrer.
- **Personas** :
  - **Léa** (étudiante) — suivre des cours, sur mobile comme sur ordinateur.
  - **Marc** (formateur) — rédiger en Markdown, gérer publication et inscrits.
  - **Awa** (admin) — gérer les comptes et modérer en toute sécurité.
- **Non fonctionnel** : sécurité, accessibilité RGAA, responsive, tests, CI.

# Base de données — modèle conceptuel (CP5)



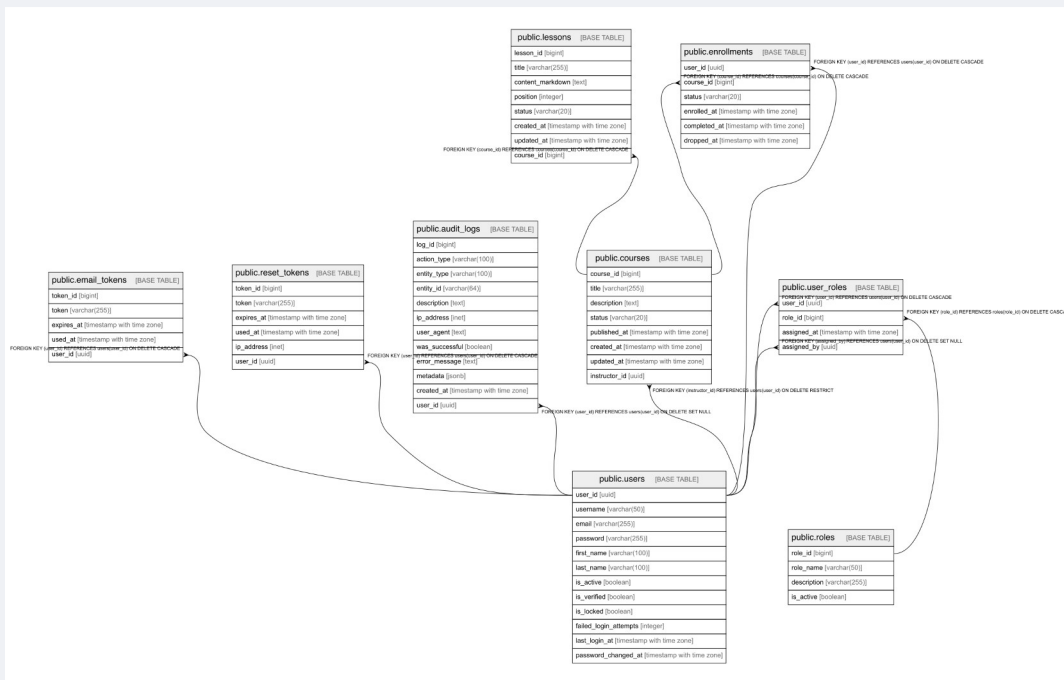


## Base de données — schéma physique & choix (CP5)

PostgreSQL 17, 9 tables, migrations Liquibase appliquées au démarrage.

Clé **UUID** pour les utilisateurs (anti-IDOR) ; **BIGINT** ailleurs ; clés composites.

Contraintes : **UNIQUE**, **CHECK**, **FK ON DELETE CASCADE** ; Hibernate en `validate`.



## Maquettage & accessibilité (CP2)

**Maquettes Figma** + prototypes HTML ; charte, deux thèmes (Catppuccin / Soft Paper).

**Responsive** : versions web (bureau) **et web mobile** (points de rupture en rem).

**Accessibilité RGAA / WCAG AA** : structure sémantique, *skip link*, contrastes calculés (min. 4,73:1), police **Atkinson Hyperlegible**.

## Courses

### Python for curious minds

By demo-teacher · You are enrolled

Loops, lists, and a taste of the standard library.

### Git from the ground up

By demo-teacher

Commits, branches, and how to think in snapshots

learn-dev, an interactive programming learning platform.

[Source on GitHub](#)

# Environnement technique (CP1)

- **Java 21 + Spring Boot 3.5** (Web MVC, Security, Data JPA, Validation, Mail).
- **Thymeleaf** (vues), **PostgreSQL 17 + Liquibase**, **commonmark-java + jsoup**.
- **Podman / Docker Compose** (Postgres, Mongo, Mailpit) ; **Maven ; Git ; CI GitHub Actions**.
- Architecture **en couches** : Contrôleur → Service → Repository → PostgreSQL.

**DÉVELOPPEMENT**

## Deux fonctionnalités CRUD

- **Feature 1 — Inscription d'un utilisateur** (validation, sécurité, jeu d'essai).
- **Feature 2 — Gestion des cours et leçons** (cycle de vie, autorisations).
- Motif commun : **Contrôleur → Service (règles métier) → Repository**, erreurs HTTP explicites (403 / 404 / 409).

## Feature 1 — Inscription (CP4, CP7)

Formulaire **validé** (Bean Validation) avec **retours d'erreur accessibles**.

**Unicité** du nom d'utilisateur et de l'e-mail ; **hachage BCrypt** du mot de passe.

Motif **Post-Redirect-Get** ; e-mail de vérification.

[learn-dev](#)[Home](#)[Log in](#)[Sign up](#)

Your registration could not be completed.  
Check the highlighted field below.

### Create an account

**Username**  
3 to 59 characters.

**Email**

Email already registered

**Password**  
At least 12 characters.

Create my account

Already registered? [Log in](#).

learn-dev — an interactive programming learning platform.[Source on GitHub](#)

# Jeu d'essai — Inscription

| Scénario                      | Attendu                                               | Obtenu   |
|-------------------------------|-------------------------------------------------------|----------|
| Cas nominal                   | Compte créé, mot de passe <b>haché</b> , rôle STUDENT | Conforme |
| E-mail déjà utilisé           | Erreur de champ, <b>aucune</b> création               | Conforme |
| Mot de passe trop court (< 8) | Rejet par la validation                               | Conforme |
| E-mail invalide               | Rejet par la validation                               | Conforme |

## Feature 2 — Cours & leçons (CP7)

**Cycle de vie** : DRAFT → PUBLISHED → ARCHIVED ; transitions invalides → **409**.

**Autorisations** : le cours d'un autre formateur → **403** ; contenu invisible → **404**.

Réordonnancement des leçons, gestion des inscrits, **journal d'audit**.

**learn-dev**

DashboardCourses**Instructor**Log out

Lesson published.

### Edit the course

Status: Draft

**Title**

Up to 255 characters.

**Description**

Optional. Shown to students in the catalogue.

Loops, lists, and a taste of the standard library.

Save the course

Publish the course

Archive the course

[View the students of this course](#)

### Lessons

Add a lesson

1. [Lists in five minutes](#) (Published)

Move downArchive

2. [Loops without tears](#) (Draft)

Move upPublishArchive

[Back to my courses](#)

learn-dev, an interactive programming learning platform.

[Source on GitHub](#)

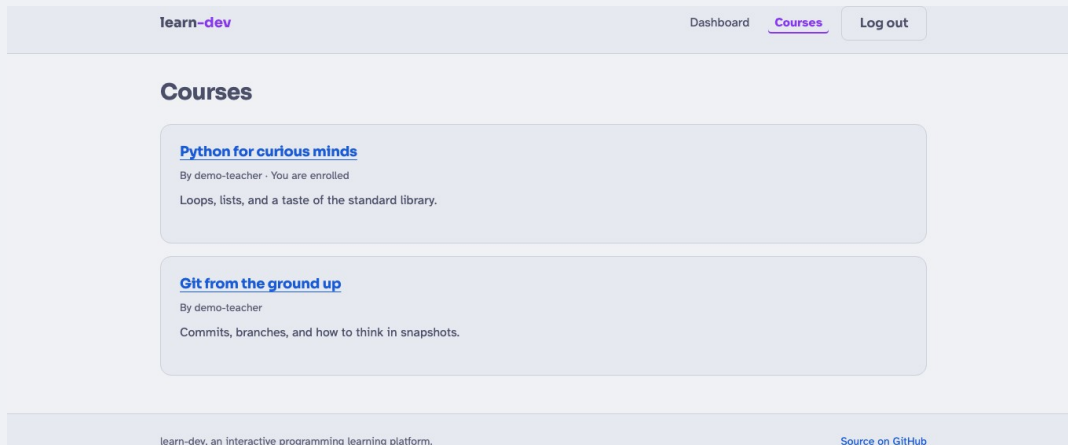


## Front-end : interfaces & responsive (CP3, CP4)

21 gabarits Thymeleaf, HTML5 sémantique, CSS responsive, fragment de mise en page partagé.

Rendu **dynamique** du Markdown des leçons (converti puis **assaini** contre les XSS).

Navigation précédent/suivant, affichage conditionnel selon le rôle.



## Accès aux données (CP6)

- **Spring Data JPA** : *repositories* = interfaces, requêtes **dérivées** et **paramétrées** (anti-injection SQL).
- **@EntityGraph** pour charger un utilisateur et ses rôles en une requête ; **@Transactional**.
- **NoSQL** : MongoDB **provisionné** en vue d'un stockage de contenu (non encore branché).

# Sécurité

- **BCrypt** (mots de passe), **CSRF** activé, **anti-XSS** (assainissement jsoup).
- **Anti-IDOR** (UUID), **anti-énumération**, **verrouillage de compte**, **rate-limiting**.
- Jetons **hachés SHA-256**, **journal d'audit**, matrice d'autorisations testée.
- Référence **OWASP Top 10** ; veille **Semgrep**.

# Tests

- **64 tests** (pyramide) : unitaires (Mockito), tranche JPA, intégration (MockMvc).
- **PostgreSQL réel** en test via **Testcontainers** (schéma fidèle : UUID, contraintes).
- **Couverture** mesurée par **JaCoCo**, publiée sur **Codecov** ; CI à 4 workflows.

**DÉPLOIEMENT**

# Conteneurisation & CI/CD (CP8)

- **Image Docker multi-étapes** : build Maven puis exécution **non-root**.
- **Docker Compose** (Postgres / Mongo / Mailpit) ; profils **dev** / **prod** ; secrets via `.env`.
- **Procédure documentée** : `docker compose up -d` puis `./mvnw spring-boot:run`.
- **CI GitHub Actions** : build, tests, lint, contrôle anti-dérive de schéma.

## CONCLUSION

# Bilan des compétences

| CP  | Compétence               | Où                             |
|-----|--------------------------|--------------------------------|
| CP1 | Environnement de travail | Stack, Docker, CI              |
| CP2 | Maquetter                | Figma, RGAA, responsive        |
| CP3 | Interfaces statiques     | Thymeleaf, CSS, thèmes         |
| CP4 | Interfaces dynamiques    | Formulaires, PRG, Markdown     |
| CP5 | Base de données          | Merise, PostgreSQL, Liquibase  |
| CP6 | Accès aux données        | Spring Data JPA (+ Mongo)      |
| CP7 | Composants métier        | Services, règles, erreurs HTTP |
| CP8 | Déploiement              | Docker, Compose, CI/CD         |



# Difficultés & évolutions

- **Difficultés** : concurrence à l'inscription (contrainte d'unicité captée) ; sécurité du rendu Markdown (XSS → assainissement) ; un seul h1 par page (accessibilité).
- **Évolutions** : brancher **MongoDB** (NoSQL) ; suivi de progression leçon par leçon ; API REST.

# Merci

- **Merci de votre attention.**
- **Projet : [github.com/ebouchut/learn-dev](https://github.com/ebouchut/learn-dev)**
- **Des questions ?**